open-cam: an open-source hyperspectral, ray-traced camera simulator built on pbrt-v4 for sensor, lens and CFA studies

Brian Deegan; University of Galway; Galway, Ireland

Abstract

We present open-cam, an open-source (MIT) camera simulator that couples the physically based renderer pbrt-v4 with a radiometrically explicit sensor model. Unlike most driving and robotics simulators, open-cam never forms an RGB image inside the renderer: pbrt-v4 writes per-wavelength radiance buckets, which are converted to sensor-plane irradiance, photon counts and photoelectrons with measured quantum efficiency and infrared-cut-filter curves, then passed through an EMVA1288-style noise, ADC and colour filter array model. Lenses can be traced through real prescriptions. We describe the architecture, compare it with open-source and commercial camera simulators, and report reproducible validation: an end-to-end ColorChecker render, a photon-transfer check that recovers the configured system gain to within 0.3%, a traced-lens slanted-edge MTF, and non-Bayer filter arrays (RGBW, RCCB, RYYCy and quad Bayer) sampled, binned and demosaiced by the pipeline from a single spectral render. We also measure the bias that pbrt-v4's path regularization introduces into caustics and reflections, and show spectral driving scenes in the visible and the short-wave infrared. We state the remaining limitations explicitly.

Introduction

Camera simulation is used to prototype sensors, to evaluate image signal processors (ISPs) and to generate training and test data for perception. Two families of tools exist. Image-systems simulators such as ISETCam [1, 2, 3] model the physics of light, optics and sensors in spectral units, while driving and robotics simulators (CARLA [4], AirSim [5], NVIDIA Isaac Sim [6] and commercial platforms) emphasise real-time scene dynamics and typically render RGB images to which camera effects are added afterwards.

Many questions that matter in automotive and mobile imaging cannot be answered from an RGB render: how a red–clear–clear–blue (RCCB) or red–yellow–yellow–cyan (RYYCy) mosaic changes low-light signal-to-noise ratio (SNR), how the infrared-cut filter (IRCF) interacts with an LED or tungsten illuminant, or how a lens with longitudinal chromatic aberration blurs each colour channel. All require the spectral radiance at the sensor. open-cam [7] is a Python toolkit built to answer such questions with free tools. Its contributions are: (i) a spectral pipeline from pbrt-v4 [8, 9] spectral film to photoelectrons, with every radiometric factor explicit; (ii) an EMVA1288-style [10] sensor model with analytic validators and real-renderer end-to-end tests; (iii) a library of camera recipes, measured spectral data and traced lens prescriptions; (iv) generic periodic colour filter arrays (CFAs), sampled per site from the spectral render; and (v) a measured account of renderer bias and of what is not yet implemented.

Related work

Image-systems simulation. ISETCam (MATLAB, MIT licence) keeps scene spectral radiance, optical-image irradiance and sensor photoelectrons and voltages in physical units, and includes models of CFAs, optics and displays [3, 2]. ISET3D (MIT) adds 3D rendering with pbrt-v4 [11]; it has been validated against measured cameras [12, 13] and extended with ray-transfer functions for lenses whose design is hidden [14]. ISETAuto (Apache-2.0) applies the stack to driving scenes and network evaluation [15, 16, 17]. open-cam is closest to this family; it differs in being pure Python, in its configuration-driven catalogue of 73 camera recipes, and in its EMVA1288-oriented validators and continuous-integration tests that run the real renderer. It does not yet offer the breadth of ISETCam's optics, display and human-vision models.

Renderers. pbrt-v4 (Apache-2.0) renders with spectral sampling and provides a `spectral` film that stores radiance in wavelength buckets, as well as a `realistic` camera that traces a lens prescription [8, 18, 19]. Mitsuba 3 (BSD-style licence) offers spectral and polarised variants [20, 21]. Both are renderers, not camera models: they stop at radiance or at a colour image.

Driving and robotics simulators. CARLA (MIT) renders RGB cameras with Unreal Engine post-processing; its documented camera attributes include auto exposure, bloom, grain jitter and lens-distortion parameters [22]. AirSim (MIT) exposes noise and distortion settings for its RGB cameras [23]; Gazebo models camera noise as independent Gaussian noise per pixel [24]. NVIDIA Isaac Sim's RTX camera documents OpenCV pinhole and fisheye distortion models, additive noise, encoding of the RGB image into a single-channel Bayer mosaic and an on-chip ISP stage [6]; NVIDIA's DRIVE simulation stack advertises high-fidelity sensor simulation [25]. None of these documentations describe spectral rendering into a QE-weighted sensor.

Commercial sensor and optics tools. Ansys Speos performs spectral, wavelength-dependent ray tracing with camera and detector response simulation [26, 27]; Ansys AVxcelerate Sensors provides a real-time, physics-based camera model parameterised from a camera datasheet and a multispectral material database [28]. Lens-design packages (Ansys Zemax OpticStudio [29], Synopsys LightTools [30]) provide sequential and non-sequential ray tracing and stray-light analysis. Scenario-level platforms, including Siemens Simcenter Prescan [31], IPG CarMaker [32], Hexagon VTD [33], rFpro [34], Cognata [35] and dSPACE AURELION, describe "physics-based" or ray-traced sensor simulation; rFpro, for example, documents a multi-exposure HDR camera API with motion blur [34]. Foretellix [36] targets scenario generation and coverage rather than sensor physics. For these commercial tools the spectral resolution, noise model and CFA options are not specified in the public material we could access, so Table 1 marks them as not publicly documented rather than guessing. Pricing is quote-based throughout.

Measurement. EMVA1288 [10, 37], photon transfer [38] and ISO 12233 slanted-edge analysis [39, 40] define how real cameras are characterised; the EMVA1288 reference implementation (LGPL-3.0) [41] and commercial suites such as Imatest [42] implement them for measured data. open-cam uses the same definitions so that simulated and measured cameras can be compared on equal terms.

Architecture

open-cam is a set of command-line tools orchestrated by `tools/run_pipeline.py`. A single YAML file (`config/pipeline.yaml`) selects a camera recipe and drives four stages: scene generation, pbrt-v4 rendering, spectral-to-electron conversion and the sensor/noise/CFA stage, followed by validators. A recipe in `config/camera_recipes/` (73 are provided: phone, compact, interchangeable-lens and research cameras) references a sensor model (QE curves, pixel pitch, fill factor, exposure, EMVA parameters, CFA) and a lens model. Sensor models are written as deltas from `config/sensor_models/default.yaml`, so a new sensor needs only its overrides.

Scenes. `build_colorchecker_scene.py` builds a 24-patch ColorChecker from measured X-Rite reflectances lit by a tabulated illuminant (D50–D75, A, C, fluorescent and LED spectra are included). `build_image_quality_targets.py` generates slanted-edge, Siemens-star and ISO-style noise targets, and `build_munsell_scenes.py` builds charts from 1269 measured Munsell chips (380–800 nm at 1 nm).

Spectral rendering. Scenes are rendered with pbrt-v4's `spectral` film, which writes an OpenEXR file with one radiance channel per wavelength bucket (`S0.`$\langle\lambda\rangle$`nm`, in $\mathrm{W\,m^{-2}\,sr^{-1}\,nm^{-1}}$) [18]. The default pipeline uses 64 buckets between 360 and 830 nm. Cameras can be pinhole, thin-lens or pbrt-v4's `realistic` camera, which traces rays through a lens prescription [19]. The repository ships four prescriptions (a 22 mm wide-angle, a 50 mm double-Gauss, a 250 mm telephoto and a 10 mm fisheye) and 14 lens models that bind them, or thin-lens parameters, to camera classes.

Radiance to electrons. Radiance is converted to sensor-plane irradiance by `pbrt_spectral_exr_to_electrons.py` using the camera equation

$$E_\lambda(x,y) = L_\lambda(x,y)\,\frac{\pi\,\tau(\lambda)}{4N^2(1+m)^2}\,c_{\mathrm{cal}}, \tag{1}$$

where N is the f-number (taken from the traced prescription when the realistic camera is used), m the magnification, τ the optics transmittance and c_{cal} an optional photometric calibration to a target scene illuminance. The mean electrons in channel c are then

$$\mu_{e,c} = \sum_\lambda E_\lambda \frac{\lambda}{hc}\,\eta_c(\lambda)\,T_{\mathrm{IR}}(\lambda)\,\Delta\lambda\;A_{\mathrm{px}}\,t\,F\,V_c(x,y), \tag{2}$$

with QE η_c, IRCF transmittance T_{IR}, bucket width $\Delta\lambda$, pixel area A_{px}, exposure time t, fill factor F and an optional vignetting map V_c. The photon conversion, QE, bin widths and geometry are folded into a $K \times C$ weight matrix (`sensor_radiometry.spectral_electron_weights`), so integration is a single matrix product per pixel. The same helper is shared with `spectral_sensor_forward.py`, an analytic forward model used for fast tests. QE curves for many cameras are taken from a measured spectral-sensitivity database; generic red, green, blue, cyan, yellow, magenta and monochrome curves and an IRCF are also provided.

Sensor model. `apply_emva_noise.py` implements the EMVA1288 linear camera model [10]: Poisson shot noise on signal plus dark current (with a temperature model), PRNU as a multiplicative gain map, DSNU as an additive offset map, Gaussian read noise, optional kTC noise, row/column fixed-pattern and flicker noise, hot/stuck pixels, blooming, inter-pixel crosstalk, full-well clipping and an ADC with gain K ($\mathrm{e^-/DN}$), black level, bit depth and optional INL/DNL. Spatial maps use a fixed seed (one "camera unit") while temporal noise uses a per-frame seed. A Bayer mosaic is sampled with any of the four RGGB/BGGR/GRBG/GBRG phases and demosaiced bilinearly or with the Malvar–He–Cutler algorithm [43]. `apply_spectral_psf.py` adds post-render optics where a traced lens is not used: Gaussian, wavelength-dependent Gaussian and Airy-disk PSFs per spectral bucket, lateral chromatic aberration and stray light.

Validation and tests. `validate_emva_model.py` compares the analytic EMVA mean and variance with Monte Carlo at photon-transfer levels and checks the recipe against its datasheet block; `batch_validate_emva.py` runs this across all recipes. `validate_demosaic_linear.py` checks demosaic fidelity in linear DN. Twenty test modules include two that call the real pbrt-v4 binary: `test_pbrt_e2e.py` and `test_pbrt_full_pipeline.py`. The latter renders a 96×64, 16-bucket ColorChecker and, for every pixel, forms the residual $z = (\mathrm{DN} - (b + \mu_e/K))/\sigma_{\mathrm{pred}}$ against the analytic EMVA prediction, requiring zero mean and unit variance; it also checks ADC limits, seed determinism and demosaic error.

Teaching. A Dear PyGui [44] desktop application (`gui/`) exposes the same engines through interactive scenarios on optics, PSF and aberrations, MTF, sensor noise, exposure and defects, colour/ISP and image generation, with accompanying tutorials.

Results

All results below were produced on a CPU build of pbrt-v4 with the scripts in `paper/ei2027/scripts/`; the exact commands are listed in the accompanying README. Renders are 960×640.

End-to-end ColorChecker. We rendered the ColorChecker with the geometry and lens of `config/pipeline.yaml` (realistic 22 mm wide-angle prescription, 8.756 mm aperture, focus 4 m, 1024 samples per pixel, 64 buckets over 360–830 nm) under CIE D65 and CIE A, and ran it through the `iphone_8` recipe (1.4 µm pixels, GBRG, Malvar demosaic, $K = 9.906\,\mathrm{e^-/DN}$, $9500\,\mathrm{e^-}$ full well, 10-bit) at 500 lux and 0.12 s, the pipeline defaults. With the realistic camera the radiometry uses the traced on-axis working f-number of the prescription at this aperture, f/2.79, rather than the nominal focal length over aperture diameter (f/2.51). The white patch yields 3.0/6.8/3.6 $\mathrm{ke^-}$ (R/G/B) under D65. Fig. 1 shows the demosaiced output with grey-world preview white balance; the gains change from 2.18/1.00/1.84 (D65) to 1.26/1.00/3.23 (A), a direct consequence of integrating each illuminant's spectrum against the camera QE rather than of an RGB tint. Against a noise-free reference, `validate_demosaic_linear.py` reports a Malvar demosaic error of 2.6 DN MAE and 5.7 DN RMSE on the 10-bit output, whose white-patch green signal is about 680 DN.

Photon transfer. Using the iphone_8 EMVA parameters ($\sigma_d = 1.6\,\mathrm{e^-}$, PRNU 1.8%, DSNU $0.16\,\mathrm{e^-}$), we simulated 256×256 uniform frame pairs at 29 exposure levels from dark to beyond saturation with `emva_theory.simulate_uniform_stack`, and applied the EMVA1288 photon-transfer method (temporal variance from frame-pair differences; zero-intercept fit of $\sigma_y^2 - \sigma_{y,\mathrm{dark}}^2$ against $\mu_y - \mu_{y,\mathrm{dark}}$ up to 70% of full well). The fit recovers $K = 9.883\,\mathrm{e^-/DN}$ against

Camera-simulation capabilities as stated in each tool's public documentation or papers (accessed October 2026). "n.d." = not documented in the public sources we reviewed; it does not mean the capability is absent.

Tool	Spectral rendering	Ray-traced lens	Sensor noise / EMVA1288	CFA flexibility	Licence / cost
open-cam	Yes: pbrt-v4 spectral film, user-set buckets	Yes: pbrt-v4 realistic camera, 4 prescriptions	Shot, read, PRNU, DSNU, dark current, ADC, FPN, defects; EMVA validators	Any periodic tile of QE curves (RGBW, RCCB, RYYCy, quad Bayer); binning	MIT; free
ISETCam / ISET3D / ISETAuto	Yes (spectral radiance)	Yes (pbrt-v4; ray-transfer functions)	Physical photoelectron/voltage model	CFA models included [3]	MIT / Apache-2.0; MATLAB required
pbrt-v4, Mitsuba 3	Yes	pbrt-v4: realistic camera	None (renderer)	None	Apache-2.0 / BSD; free
CARLA, AirSim, Gazebo	No (RGB)	No; distortion parameters	Grain / Gaussian / noise-line effects	None (RGB output)	MIT / Apache-2.0; free
Isaac Sim (RTX)	n.d.	No; OpenCV distortion models	Additive noise	Bayer encoding + ISP	NVIDIA licence
Ansys Speos	Yes	Yes	Detector response, raw signal	n.d.	Commercial
Ansys AVxcelerate	Multispectral materials	n.d.	Datasheet-parameterised	n.d.	Commercial
Zemax OpticStudio, LightTools	Wavelength-dependent tracing	Yes (lens design)	n.d.	n.d.	Commercial
Prescan, CarMaker, VTD, rFpro, Cognata, AURELION, DRIVE Sim	n.d.	n.d. (rFpro: ray tracing)	n.d.	n.d.	Commercial
Imatest, emva1288	– (measurement)	–	EMVA1288 analysis of measured data	–	Commercial / LGPL-3.0

the configured 9.906 (0.23% error) and $\sigma_d = 1.588\,\mathrm{e}^-$ (configured 1.6). The EMVA PRNU estimate from 16-frame dark and 50%-saturation stacks is 1.79% (configured 1.8%). Across the linear range the simulated variance deviates from the analytic model by at most 1.3% (median 0.46%), consistent with the sampling error of a 65,536-pixel estimate (Fig. 2). `validate_emva_model.py` passes all photon-transfer, dark-frame and datasheet checks for this recipe with 20,000 Monte Carlo trials.

Traced-lens MTF. We rendered a 5° slanted edge at 3.2 m through the 50 mm double-Gauss prescription (32 buckets, 400–700 nm, 4096 samples per pixel) at f/2 in focus, at f/2 focused at 2.6 m, and at f/5.6 focused at 2.6 m, and analysed it with `sfr_analysis.slanted_edge_sfr`. With the green QE×IRCF of the default research camera the MTF50 is 0.257, 0.328 and 0.339 cy/px respectively, and the in-focus MTF at Nyquist is 0.267 (Fig. 3). Two observations are worth reporting as measured rather than explained away. First, the nominal `focusdistance` of 3.2 m does not maximise MTF50: focusing at 2.6 m gives a higher MTF50, but the two f/2 curves cross near 0.4 cy/px and the nominal focus keeps more contrast towards Nyquist (0.267 against 0.190). This is consistent with residual spherical aberration at f/2, where moving focus trades a sharp core for a smaller halo, though we have not verified that cause. Best focus therefore depends on the frequency of interest and should be found with a through-focus sweep, not the nominal value. Second, stopping down to f/5.6 raises MTF50 further, as expected from the smaller blur circle. Because the film is spectral, the same render also yields an MTF per wavelength; Fig. 3 (right) shows MTF50 in bands of four buckets (37.5 nm). open-cam traces the glass dispersion of the prescription (the patent's ν_d values, with the $n(\lambda)$ shape of the nearest SCHOTT glass), and the bands show the resulting axial colour. At f/2 in focus, MTF50 is highest near 490–530 nm and falls towards both ends of the band (0.18–0.27 cy/px); at f/5.6 focused at 2.6 m the curve is U-shaped (0.33–0.46 cy/px), consistent with blue and red foci lying closer to the edge distance than the green focus. With dispersion switched off (`-no-lens-dispersion`), an otherwise identical f/2 in-focus render gives an MTF50 of 0.237–0.243 cy/px in every band, so the variation comes from the glass model, not the renderer. An earlier version of this analysis showed a periodic ripple across buckets; it was a pbrt-v4 wavelength-sampling artefact (one wavelength offset shared by the four hero wavelengths, and a seed override), which open-cam now removes by jittering each wavelength stratum independently. Edge positions were located with a 50%-crossing fit after per-row flat-fielding, because the derivative-centroid estimator in `sfr_analysis.py` was biased by render noise on this edge (it reported a −2.3° edge for a measured −5.0°); the ESF binning, LSF and MTF use the repository functions. These values are in film pixels and include the pixel aperture: each pixel integrates radiance over its own area (a box reconstruction filter one pixel wide), whose MTF $|\mathrm{sinc}\, f|$ is 0.64 at Nyquist. Dividing it out gives a lens-only in-focus MTF at Nyquist of 0.42.

Colour filter arrays. The pipeline samples any periodic tile of named channels. Each channel has its own QE curve, which is multiplied by the IR-cut filter and integrated against the spectral buckets at each site, so a clear pixel sees the unfiltered silicon QE rather than a substitute curve. Shot noise, PRNU, DSNU, dark current, full well and ADC act per site. Bayer layouts keep the bilinear and Malvar–He–Cutler demosaics; other layouts use bilinear, gradient-guided or edge-directed interpolation, and quad Bayer [45] can be

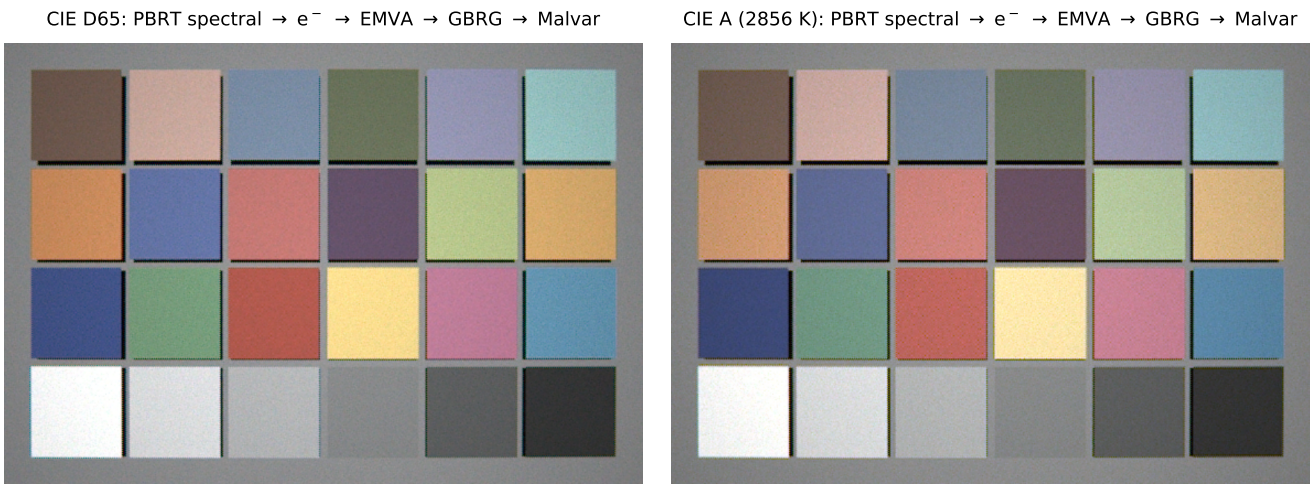


Figure 1. *ColorChecker rendered spectrally by pbrt-v4 (64 buckets) and processed by open-cam with the iphone_8 recipe: electrons → EMVA noise → GBRG mosaic → Malvar demosaic, grey-world preview white balance, no colour correction. Left: D65; right: CIE A.*

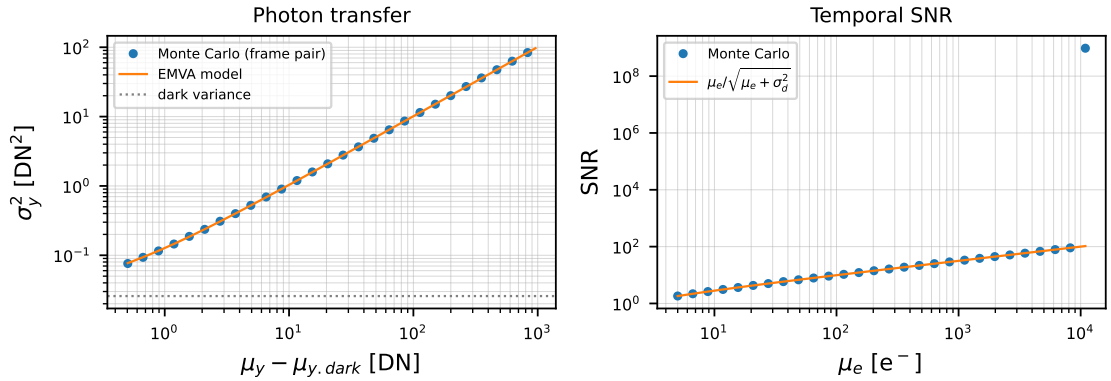


Figure 2. *EMVA1288 photon-transfer validation for the iphone_8 recipe. Left: temporal variance versus dark-subtracted mean signal for Monte Carlo frame pairs (markers) and the analytic model (line); the gain fit uses levels up to 70% of full well. Right: temporal SNR versus mean electrons.*

charge- or digitally binned 2×2 or remosaiced to Bayer. We ran six recipes of the default research camera (Fig. 4) through the pipeline on the D65 and CIE A ColorChecker renders at 50 lux and 0.12 s, the level at which the white patch stays below ADC full scale for every layout; with 2×2 charge binning the 10-bit ADC, not the quadrupled full well, limits the binned signal to 4.8 ke$^-$. The renders cover 360–830 nm, so the 0.1–0.3% leak of the IR-cut filter beyond 830 nm is not counted.

Table 2 reports, for the neutral 5 patch (about 19% reflectance), the luminance-channel signal relative to the Bayer green pixel (137 e$^-$ at 50 lux under D65), the temporal SNR at 10 lux measured from two noise seeds next to the analytic EMVA value $\mu_e/\sqrt{\mu_e + \sigma_d^2}$ with $\sigma_d = 2.6\,\mathrm{e}^-$, and the colour error of the layout. A clear or white pixel collects 2.52–2.54 times the green signal and gains 4.6–5.1 dB of SNR at 10 lux; 2×2 charge binning collects 4.05 times the green signal at a quarter of the resolution and gains 7.3 dB; yellow gains 2.0 dB. The measured SNR agrees with the analytic value to within 0.6 dB for every layout. The yellow/green ratio depends on the illuminant, rising from 1.46 under D65 to 1.96 under CIE A, while the clear/green ratio stays at 2.50–2.51. The colour error is the mean CIEDE2000 of the 24 patches after a least-squares colour-correction matrix fitted to their noise-free spectral responses under D65; it measures how well each set of channels spans colour, and excludes the noise that the matrix amplifies, which Fig. 5 shows is clearly visible for RGBW at 50 lux. Per-site analyses of this kind need the spectral radiance at each pixel and cannot be derived from an RGB render [46].

Rendering bias from path regularization

pbrt-v4's path tracer offers a `regularize` option: after the first non-specular bounce, every later near-specular surface is treated as rough (GGX roughness raised to between 0.1 and 0.3). This removes the rare, very bright samples that make caustics and reflected highlights noisy, but it changes the light transport, so the estimate converges to a different image. More samples reduce the noise but not this bias. Our scene builders used regularization until we measured it; they now use the unregularized path tracer by default and keep regularization as an option.

We measured the bias by rendering small crops at very high sample counts with and without regularization, using the same spectral

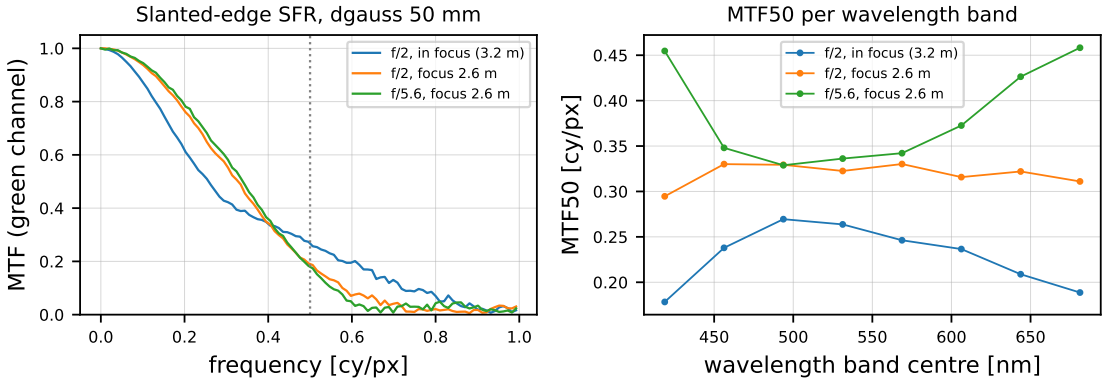


Figure 3. *Slanted-edge MTF through the traced 50 mm double-Gauss lens. Left: green-channel MTF for three aperture/focus settings (dotted line: Nyquist). Right: MTF50 of the same renders in 37.5 nm wavelength bands; the variation is the axial colour of the traced glasses (see text).*

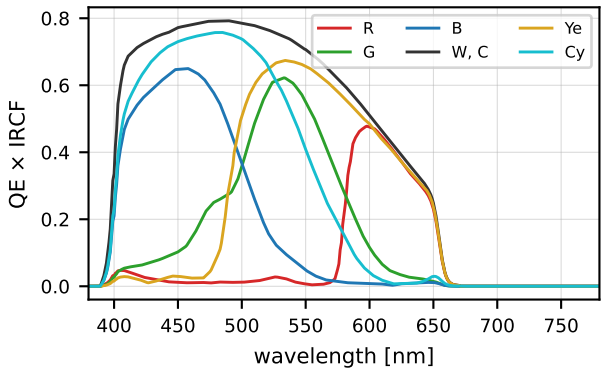


Figure 4. *Per-channel QE×IRCF curves of the default research camera used for the CFA study (W and C: unfiltered silicon QE; IRCF: hybrid BG60 glass and interference coating).*

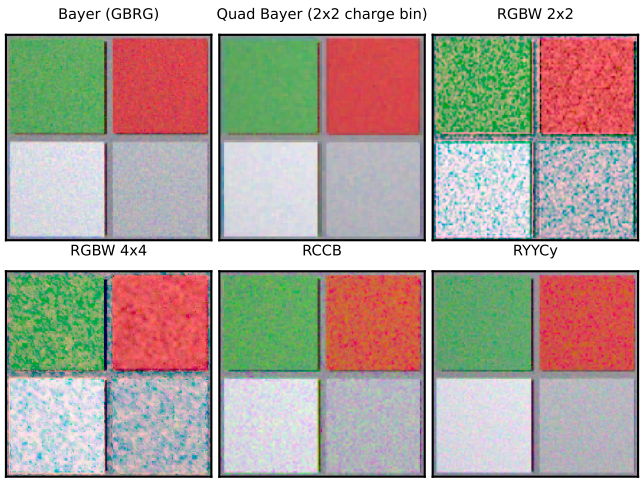


Figure 5. *Pipeline output for six CFA recipes from one spectral render (D65, 50 lux, 0.12 s) at the corner of patches 14, 15, 20 and 21: sampled, binned (quad Bayer) and demosaiced by the pipeline, white-balanced on the white patch and colour-corrected with the spectral matrix of Table 2.*

film and lens (Table 3). In the image-quality laboratory scene, regularization made the core of the caustic under a smooth glass sphere 12% too dark: 33.9 against 38.6 (relative radiance) at 16 384 samples per pixel, while ordinary renders at 1024 and 4096 samples gave 33.1 and 33.9. The floor and walls agreed to within 0.2%. In the road scenes the bias appears where light reaches a diffuse surface via water or glass. On a wet rural road at night, tree canopies lit partly by lamp light reflected off the water were 7.0% too dark, while the same pixels on a dry road were unchanged; on a wet urban road in daylight the road and parked cars were 2.8% too dark. Seen through a windscreen, the road was 1.9% too bright. The tunnel ceiling changed by 0.1%.

The unregularized path tracer is unbiased but noisy in caustics. We therefore added an optional split to pbrt-v4: the path tracer skips paths whose last bounces before the first diffuse vertex are all specular, and a photon pass with a shrinking gather radius renders exactly those paths. The two images are summed. In the laboratory scene the split agreed with the unregularized reference to within 1% (38.2 against 38.6) and reduced the per-pixel noise in the caustic from 12.1% to 6.8% at 1024 samples, for about 1.3 times the render time, while the noise on the directly lit floor was unchanged (5.24% against 5.21%). An RGB render with camera effects added afterwards could show the noise, but not attribute the missing caustic energy to the renderer.

Driving scenes

The same pipeline renders procedural road scenes. Figure 6 shows a highway with the sun-and-sky daylight model, a wet road with puddles and tyre spray, the dry scene in the short-wave infrared (SWIR), and a wet road at night, all rendered on the unregularized path tracer. For the SWIR panel the sun follows the ASTM G173 direct spectrum, including its water-vapour bands, vegetation follows a measured USGS leaf spectrum above 800 nm, and the film integrates an ideal 900–1600 nm passband; this needs a pbrt-v4 build with an extended wavelength range. These renders demonstrate the scene and weather models; we have not validated them against measured road imagery.

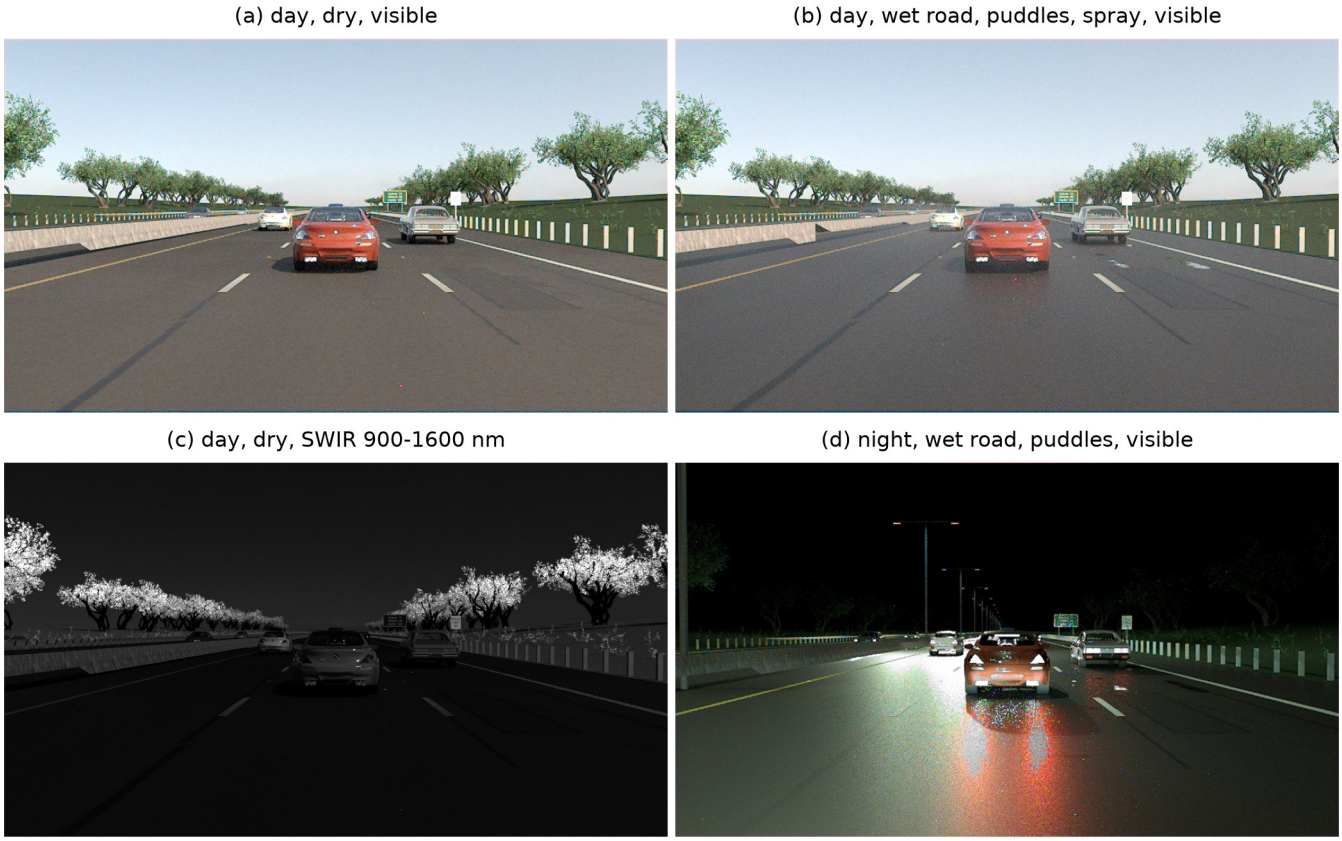


Figure 6. *Highway scenes, 960×540, unregularized path tracer, 256 samples per pixel for (a)–(c) and 1024 for (d). (a), (b), (d): pbrt-v4 spectral film with 16 buckets over 360–830 nm, converted by the default research-camera recipe and shown as its noisy demosaiced sRGB preview. (c): 36 buckets over 900–1600 nm, converted by a representative 15 µm InGaAs recipe and shown as its noisy greyscale preview. Exposure is set separately for each panel (99th percentile at 80% of full well), so brightness is not comparable between panels. The bright speckle on the wet road just ahead of the lead car in (d) is Monte Carlo noise in the pbrt-v4 output: ego low-beam light reflected by the lead car and then by the wet road, a two-bounce path that the unregularized path tracer renders without bias but still with high variance at 1024 samples per pixel.*

CFA comparison on the neutral 5 patch (default research camera, D65 and CIE A renders, 0.12 s): luminance-channel signal relative to Bayer green, temporal SNR at 10 lux measured from two noise seeds (analytic EMVA value in brackets), SNR gain over Bayer, and mean CIEDE2000 of the 24 patches after a noise-free spectral colour-correction fit.

CFA (channel)	ch/G D65	ch/G A	SNR [dB]	Δ [dB]	ΔE_{00}
Bayer (G)	1.00	1.00	13.3 (13.4)	–	1.07
Quad, 2×2 bin (G)	4.05	4.04	20.7 (20.1)	+7.3	1.07
RGBW 2×2 (W)	2.52	2.51	18.0 (18.0)	+4.6	0.93
RGBW 4×4 (W)	2.54	2.50	18.4 (18.0)	+5.1	0.93
RCCB (C)	2.54	2.51	18.0 (18.0)	+4.7	1.10
RYYCy (Ye)	1.46	1.96	15.3 (15.3)	+2.0	1.15

Bias of pbrt-v4 path regularization: mean radiance of a crop rendered with regularization divided by the unregularized reference, raw pbrt-v4 spectral output (no sensor model). Uncertainties are the standard error over independent seeds.

Scene and crop (samples per pixel)	Ratio
Laboratory, caustic core under glass sphere (16 384)	0.878
Rural night, wet road, lamp-lit canopy (16 384)	0.930 ± 0.005
Rural night, dry road, same pixels (16 384)	1.000
Urban day, wet road, road and cars (8192)	0.972 ± 0.000
Highway day, road through windscreen (8192)	1.019 ± 0.001
Tunnel, ceiling (8192)	0.999 ± 0.000

Discussion and limitations

The results support the central design choice: keeping light spectral until the photon-counting step makes illuminant, IRCF and filter questions first-class, and the EMVA validators make the noise model auditable against datasheets. The limitations are equally concrete. The generic-CFA demosaicing methods are classical (bilinear, gradient-guided and edge-directed interpolation), and the quad-Bayer remosaic is a reference implementation, not a vendor algorithm. Many recipes carry EMVA parameters inferred from public tables (each recipe records a calibration tier) rather than measured reports. Rendering is offline: the 1024-sample, 64-bucket ColorChecker took about 6 min on an 8-core CPU, so open-cam complements rather than replaces real-time simulators. The driving scenes are procedural and have not been validated against measured road imagery. HDR pixel architectures (dual conversion gain, split pixel, LOFIC, multi-exposure), exposure-time motion blur and a rolling-shutter line time are implemented but are not evaluated in this paper. Removing path regularization makes reflections and caustics noisier, so wet-road night scenes need more samples per pixel.

Conclusion

open-cam provides a free, scriptable path from pbrt-v4 spectral radiance to EMVA-consistent raw data through traced lenses and measured QE curves. It recovers configured sensor parameters through the EMVA1288 photon-transfer method, reproduces illuminant-dependent white balance from first principles, samples RGBW, RCCB, RYYCy and quad-Bayer mosaics per site from the same spectral render, and quantifies a renderer bias that an RGB pipeline could not attribute. Code, configurations and all scripts used for this paper are available at `https://github.com/briandeegan82/open-cam`.

References

[1] J. E. Farrell, F. Xiao, P. B. Catrysse, and B. A. Wandell, "A simulation tool for evaluating digital camera image quality," in *Proc. SPIE 5294, Image Quality and System Performance*, pp. 124–131, 2003.

[2] J. E. Farrell, P. B. Catrysse, and B. A. Wandell, "Digital camera simulation," *Applied Optics*, vol. 51, no. 4, pp. A80–A90, 2012.

[3] "ISETCam (MIT licence)." `https://github.com/ISET/isetcam`. Accessed 2026-10-08.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annual Conference on Robot Learning (CoRL), PMLR 78*, pp. 1–16, 2017.

[5] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, Springer Proceedings in Advanced Robotics, 2017.

[6] NVIDIA, "Isaac Sim documentation: Camera sensors." `https://docs.isaacsim.omniverse.nvidia.com/latest/sensors/isaacsim_sensors_camera.html`. Accessed 2026-10-08.

[7] B. Deegan, "open-cam: spectral pbrt-v4 camera simulation." `https://github.com/briandeegan82/open-cam`. Accessed 2026-10-08.

[8] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*. Cambridge, MA: MIT Press, 4th ed., 2023.

[9] M. Pharr *et al.*, "pbrt-v4 source code (Apache-2.0)." `https://github.com/mmp/pbrt-v4`. Accessed 2026-10-08.

[10] European Machine Vision Association, "EMVA Standard 1288: Standard for Characterization of Image Sensors and Cameras, Release 4.0." `https://www.emva.org/standards-technology/emva-1288/`, 2021. Accessed 2026-10-08.

[11] "ISET3D: pbrt-v4 based 3D rendering for ISETCam (MIT licence)." `https://github.com/ISET/iset3d`. Accessed 2026-10-08.

[12] Z. Lyu, K. Kripakaran, M. Furth, E. Tang, B. Wandell, and J. Farrell, "Validation of image systems simulation technology using a Cornell box," in *Proc. IS&T Electronic Imaging: Imaging Sensors and Systems*, 2021.

[13] Z. Lyu, T. Goossens, B. A. Wandell, and J. Farrell, "Validation of physics-based image systems simulation with 3-d scenes," *IEEE Sensors Journal*, vol. 22, no. 20, pp. 19400–19410, 2022.

[14] T. Goossens, Z. Lyu, J. Ko, G. C. Wan, J. Farrell, and B. Wandell, "Ray-transfer functions for camera simulation of 3D scenes with hidden lens design," *Optics Express*, vol. 30, no. 13, p. 24031, 2022.

[15] "ISETAuto (Apache-2.0 licence)." `https://github.com/ISET/isetauto`. Accessed 2026-10-08.

[16] H. Blasinski, J. Farrell, T. Lian, Z. Liu, and B. Wandell, "Optimizing image acquisition systems for autonomous driving," in *Proc. IS&T Electronic Imaging: Photography, Mobile, and Immersive Imaging*, 2018.

[17] Z. Liu, T. Lian, J. Farrell, and B. A. Wandell, "Soft prototyping camera designs for car detection based on a convolutional neural network," in *Proc. IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2019.

[18] "pbrt-v4 User's Guide and File Format." `https://pbrt.org/fileformat-v4`. Accessed 2026-10-08.

[19] C. Kolb, D. Mitchell, and P. Hanrahan, "A realistic camera model for computer graphics," in *Proc. SIGGRAPH '95*, pp. 317–324, 1995.

[20] M. Nimier-David, D. Vicini, T. Zeltner, and W. Jakob, "Mitsuba 2: A retargetable forward and inverse renderer," *ACM Transactions on Graphics*, 2019.

[21] "Mitsuba 3 documentation: Choosing variants." `https://mitsuba.readthedocs.io/en/stable/src/key_topics/variants.html`. Accessed 2026-10-08.

[22] "CARLA documentation: Sensors reference." `https://carla.readthedocs.io/en/latest/ref_sensors/`. Accessed 2026-10-08.

[23] "AirSim documentation: Settings." `https://microsoft.github.io/AirSim/settings/`. Accessed 2026-10-08.

[24] "Gazebo Classic tutorial: Sensor noise model." `https://classic.gazebosim.org/tutorials?tut=sensor_noise`. Accessed 2026-10-08.

[25] NVIDIA, "Simulation for Self-Driving Vehicles." `https://www.nvidia.com/en-us/self-driving-cars/simulation/`. Accessed 2026-10-08.

[26] Ansys, "Ansys Speos." `https://www.ansys.com/products/optics/ansys-speos`. Accessed 2026-10-08.

[27] M. Falahati, V. Loya, S. Kalkhoran, and V. Singh, "Modeling and evaluation of ADAS camera jitter due to road-induced random vibration using Ansys Speos," in *Proc. SPIE Optical Architectures for Displays and Sensing in Augmented, Virtual, and Mixed Reality*, 2026.

[28] Ansys, "Ansys AVxcelerate Sensors." `https://www.ansys.com/products/av-simulation/ansys-avxcelerate-sensors`. Accessed 2026-10-08.

[29] Ansys, "Ansys Zemax OpticStudio." `https://www.ansys.com/products/optics/ansys-zemax-opticstudio`. Accessed 2026-10-08.

[30] Synopsys, "LightTools illumination design software." `https://www.synopsys.com/optical-solutions/lighttools.html`. Accessed 2026-10-08.

[31] Siemens, "Simcenter Prescan." `https://plm.sw.siemens.com/en-US/simcenter/autonomous-vehicle-solutions/prescan/`. Accessed 2026-10-08.

[32] IPG Automotive, "CarMaker." `https://www.ipg-automotive.com/en/products-solutions/software/carmaker/`. Accessed 2026-10-08.

[33] S. Son, W. Lee, H. Jung, J. Lee, C. Kim, H. Lee, H. Park, H. Lee, J. Jang, S. Cho, and H.-C. Ryu, "Evaluation of camera recognition performance under blockage using virtual test drive toolchain," *Sensors*, vol. 23, no. 19, p. 8027, 2023.

[34] rFpro, "Ray tracing." `https://rfpro.com/simulation-software/ray-tracing/`. Accessed 2026-10-08.

[35] Cognata, "Cognata." `https://www.cognata.com/`. Accessed 2026-10-08.

[36] Foretellix, "Foretellix." `https://www.foretellix.com/`. Accessed 2026-10-08.

[37] M. E. Borek, "Implementation of EMVA 1288 standard release 4.0 for characterization of image sensors," in *Proc. IS&T Electronic Imaging: Imaging Sensors and Systems*, 2023.

[38] J. R. Janesick, *Photon Transfer*. SPIE Press, 2007.

[39] ISO, "ISO 12233:2017 Photography – Electronic still picture imaging – Resolution and spatial frequency responses." International Organization

for Standardization, 2017.

[40] D. Williams and P. D. Burns, "Low-frequency MTF estimation for digital imaging devices using slanted-edge analysis," in *Proc. SPIE 5294, Image Quality and System Performance*, pp. 93–101, 2003.

[41] "EMVA1288 reference implementation (LGPL-3.0)." `https://github.com/EMVA1288/emva1288`. Accessed 2026-10-08.

[42] Imatest, "Imatest image quality testing software." `https://www.imatest.com/`. Accessed 2026-10-08.

[43] H. S. Malvar, L. He, and R. Cutler, "High-quality linear interpolation for demosaicing of Bayer-patterned color images," in *Proc. IEEE ICASSP*, 2004.

[44] "Dear PyGui." `https://github.com/hoffstadt/DearPyGui`. Accessed 2026-10-08.

[45] T. Okawa, S. Ooki, H. Yamajo, M. Kawada, M. Tachi, K. Goi, T. Yamasaki, H. Iwashita, M. Nakamizo, T. Ogasahara, Y. Kitano, and K. Tatani, "A 1/2inch 48M all PDAF CMOS image sensor using 0.8μm Quad Bayer coding 2×2OCL with 1.0lux minimum AF illuminance level," in *Proc. IEEE International Electron Devices Meeting (IEDM)*, 2019.

[46] K. Hirakawa and P. J. Wolfe, "Spatio-spectral color filter array design for optimal image recovery," *IEEE Transactions on Image Processing*, 2008.

Author Biography

Brian Deegan received bachelor's (computer engineering, 2004) and MSc (biomedical engineering, 2005) degrees from the University of Limerick, and a PhD in biomedical engineering from NUI Galway (2011). From 2011 to 2022 he was a vision research engineer at Valeo Vision Systems, working on HDR imaging, LED flicker and image quality for machine vision, and from 2014 a Valeo Expert on Image Quality. In 2022 he joined the University of Galway as a lecturer in Electrical & Electronic Engineering.